SnowPro Advanced: Data Engineer — Prep Guide

PJ's Academy · The next step after SnowPro Core.

This certification (DEA-C02) proves you can **build production data pipelines** on Snowflake. It's harder than Core — scenario-based questions, deeper on performance, pipelines, and Snowpark.

> **Prerequisite:** You must hold **SnowPro Core** to earn this credential.

Exam At A Glance

Item	Detail
Exam code	DEA-C02
Questions	~65
Duration	115 minutes
Passing score	~750/1000 scaled
Cost	$375 USD
Prerequisite	SnowPro Core certified

Domain Weightings

#	Domain	Weight
1	Data Movement	25%
2	Performance Optimization	20%
3	Storage & Data Protection	10%
4	Security	10%
5	Data Transformation	25%
6	Data Pipelines & Automation	10%

Domain 1 — Data Movement (25%)

Loading deep-dive

- **COPY INTO** options: `MATCH_BY_COLUMN_NAME`, `ENFORCE_LENGTH`, `TRUNCATECOLUMNS`, `FORCE`, `PURGE`.

- **Snowpipe** internals: `AUTO_INGEST`, cloud notifications (SNS/SQS, Event Grid, Pub/Sub), `PIPE_STATUS`, load history via `COPY_HISTORY`.
- **Snowpipe Streaming** — row-level low-latency ingestion via the SDK (vs file-based Snowpipe).
- **External tables** — query files in a data lake without loading.
- **Iceberg tables** — open table format support.

Unloading

- `COPY INTO @stage`, partitioned unloads (`PARTITION BY`), `MAX_FILE_SIZE`, `SINGLE`.

Key scenario skills

- Choosing bulk vs Snowpipe vs Snowpipe Streaming for a given latency/volume requirement.
- Handling schema drift with `INFER_SCHEMA` and `MATCH_BY_COLUMN_NAME`.

⚡ Domain 2 — Performance Optimization (20%)

- **Query Profile** mastery — spilling, exploding joins, poor pruning, cartesian products.
- **Clustering** — `SYSTEM$CLUSTERING_INFORMATION`, reclustering cost, choosing keys.
- **Search Optimization Service** — cost/benefit, which predicates benefit.
- **Materialized views** — maintenance cost, when they help vs hurt.
- **Warehouse tuning** — scale up vs out, multi-cluster scaling policies (Standard vs Economy).
- **Result & metadata caching** — designing for cache hits.
- **Micro-partition pruning** — designing table/query for maximum pruning.

💾 Domain 3 — Storage & Data Protection (10%)

- Time Travel + Fail-safe cost implications; retention tuning per table type.
- Cloning strategies for CI/CD and dev/test.
- Transient vs permanent vs temporary tables (Fail-safe/Time Travel differences).
- Replication & failover groups for DR.

🔐 Domain 4 — Security (10%)

- RBAC design at scale — functional vs access roles, role hierarchies.
- Masking + row access policies with **conditional** logic and mapping tables.
- Object tagging + tag-based masking for governance at scale.
- Secure UDFs and secure views for data sharing.

🔄 Domain 5 — Data Transformation (25%)

Snowpark (heavily weighted)

- DataFrame API — lazy evaluation, `explain()`, pushdown.
- Python UDFs / UDTFs / vectorized (batch) UDFs.
- Stored procedures (Python) — orchestration logic.
- Packages, imports, and stage-based deployment.

Advanced SQL

- Recursive CTEs, `MATCH_RECOGNIZE` (pattern matching — often tested!).
- `MERGE` for upserts, multi-table inserts.
- Window frames, `IGNORE NULLS`, `PIVOT` / `UNPIVOT`.

🔁 Domain 6 — Pipelines & Automation (10%)

- **Streams** — standard, append-only, insert-only; stream staleness; multiple consumers.
- **Tasks** — serverless vs warehouse tasks, task graphs (DAGs), `FINALIZER` tasks, error handling.
- **Dynamic Tables** — declarative pipelines with target lag (the modern alternative to Streams+Tasks — know when to use each).
- **Alerts** — condition-based automated actions.

🧠 High-Yield Facts

- **Dynamic Tables** auto-refresh to a declared `TARGET_LAG` — simpler than Streams+Tasks for many pipelines.
- **Snowpipe Streaming** ≠ Snowpipe — it's row-level via SDK, lower latency, no files.
- **MATCH_RECOGNIZE** does row-pattern matching (sessionization, funnels) — memorise the syntax skeleton.
- **Vectorized Python UDFs** process batches (pandas) — faster than row-by-row.
- **Task graphs** can have a **FINALIZER** task that runs after all others (even on failure).
- **Standard stream** tracks inserts+updates+deletes; **append-only** tracks inserts only (cheaper).

📝 Practice Questions (10)

Q1. You need sub-second ingestion of individual rows from an app. Best option? A) COPY INTO on schedule B) Snowpipe (file-based) C) Snowpipe Streaming D) External table

Q2. Which declarative feature auto-maintains a transformed table to a target lag? A) Materialized view B) Dynamic Table C) Stream D) Task

Q3. Which SQL feature is designed for row-pattern matching (e.g., funnels)? A) PIVOT B) MATCH_RECOGNIZE C) QUALIFY D) FLATTEN

Q4. A stream that tracks only new inserts (not updates/deletes) is: A) Standard B) Append-only C) Insert-only D) Both B and C exist; append-only ignores deletes

Q5. Vectorized Python UDFs improve performance by: A) Running on GPU B) Processing batches (pandas) instead of row-by-row C) Caching D) Compiling to SQL

Q6. A task that runs after all tasks in a graph complete (even on failure): A) Root task B) Finalizer task C) Child task D) Serverless task

Q7. To reduce Fail-safe/Time Travel storage on a staging table: A) Permanent table B) Transient table C) Add clustering D) Materialized view

Q8. `MATCH_BY_COLUMN_NAME` in COPY INTO helps with: A) Faster loads B) Loading semi-structured columns by name (schema drift) C) Compression D) Error handling

Q9. Which shows clustering health of a table? A) QUERY_HISTORY B) SYSTEM$CLUSTERING_INFORMATION C) COPY_HISTORY D) PIPE_STATUS

Q10. Best tool to diagnose an exploding join: A) Resource monitor B) Query Profile C) Task history D) Stream

✅ Answers

1-C · 2-B · 3-B · 4-D · 5-B · 6-B · 7-B · 8-B · 9-B · 10-B

🗓️ 3-Week Plan

- **Week 1:** Data Movement + Transformation (Snowpark hands-on).
- **Week 2:** Performance + Pipelines (Dynamic Tables, Streams/Tasks labs).
- **Week 3:** Storage/Security review + practice, then book the exam.

❄️ *Snowflake Mastery — PJ's Academy*